

Criando APIs para o Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br

Sumário

- API (Application Programming Interface)
- Protocolo HTTP
- EndPoint
- JSON (JavaScript Object Notation)
- Atividade 01
- Servidor Backend e Web Service
- Criando uma API REST com Express
- Atividade 02

Métodos HTTP

- **GET**

- Finalidade: Recuperar informações do servidor
- Características: Seguro (não altera dados), idempotente (várias chamadas = mesmo resultado)

- **POST**

- Finalidade: Criar novos recursos no servidor
- Características: Não idempotente (chamadas repetidas criam múltiplos recursos)

- **PUT/PATCH**

- PUT: Substitui completamente um recurso existente
- PATCH: Atualiza parcialmente um recurso
- Características: Ambos servem para atualizar as informações

- **DELETE**

- Finalidade: Remover um recurso específico
- Características: Idempotente (apagar algo já apagado não causa erro)

EndPoint

- Um endpoint é uma URL específica que fornece acesso a um recurso ou funcionalidade em uma API.
- Representa o ponto de comunicação entre o cliente e o servidor.
- **Exemplo:** <https://github.com/awesomeapibrasil/awesomeapi-cep>
 - Método GET: Lista todos os usuários.
 - Método POST: Adiciona um novo usuário.

JSON (JavaScript Object Notation)

- Formato leve de troca de dados que:
 - É fácil para humanos ler e escrever
 - É fácil para máquinas parsear e gerar
- Baseado em dois tipos de estruturas:
 - Coleções de pares nome/valor (objetos)
 - Listas ordenadas de valores (arrays)

```
1  {  
2    "id": "USER-001",  
3    "nome": "Carlos Silva",  
4    "email": "carlos@exemplo.com",  
5    "idade": 28,  
6    "ativo": true,  
7    "endereco": {  
8      "cidade": "São Paulo",  
9      "estado": "SP"  
10   },  
11   "interesses": ["tecnologia", "esportes"]  
12 }
```

```
1  {  
2    "frutas": ["maçã", "banana", "laranja"],  
3    "numeros": [1, 2, 3, 5, 8, 13],  
4    "booleanos": [true, false, true],  
5    "objetos": [  
6      {"id": 1, "nome": "item A"},  
7      {"id": 2, "nome": "item B"}  
8    ],  
9    "vazio": []  
10 }
```

Fonte: Elaborado pelo autor.

Servidor Backend e Web Service

- Servidor Backend: Sistema que processa requisições, gerencia dados e fornece respostas para clientes (apps, navegadores).
- Funções Principais:
 - Armazenar/recuperar dados (banco de dados)
 - Executar regras de negócio
 - Fornecer APIs para comunicação

Servidor Backend e Web Service

- Web Service: Serviço acessível via web que permite comunicação entre sistemas usando HTTP/HTTPS.
- Permite que sistemas heterogêneos (diferentes linguagens, plataformas ou tecnologias) se comuniquem de forma padronizada.

Criando uma API REST com Express

- # Passo 1: Inicializando o projeto
 - Crie uma nova pasta para seu projeto e abra no VS Code.
- # Passo 2: Instale o [Express.js](#)
 - `npm install express`
- # Passo 3: Instale o Cors
 - `npm install cors express`

Criando uma API REST com Express

- # Passo 4: Criando o arquivo [api.js](#)
 - Seguir o exemplo da imagem.
- # Passo 5: Executando o servidor
 - node api.js

CORS é um mecanismo de segurança que controla acesso entre domínios diferentes no navegador.

```
JS api.js > ...
1 import express from 'express';
2 import cors from 'cors';
3
4 const app = express();
5
6 app.use(cors());
7
8 app.get('/', (req, res) => {
9   res.json({
10     date: new Date().toLocaleString('pt-BR'),
11     status: 'API no Render funcionando!'
12   });
13 });
14
15 // Porta dinâmica para o Render
16 const PORT = process.env.PORT || 3000;
17 app.listen(PORT, () => {
18   console.log(`Servidor rodando na porta ${PORT}`);
19 });
```

Fonte: Elaborado pelo autor.

Framework Express.js

- O Express.js é um framework para Node.js que facilita a criação de servidores web e APIs (interfaces de programação de aplicações).
- Ele é minimalista, flexível e muito popular no ecossistema JavaScript.

Framework Express.js

- Simplifica o Node.js: O Node.js puro (http module) exige mais código para rotas, middlewares, etc. O Express torna isso mais fácil.
- Roteamento: Facilita a definição de rotas (ex: /users, /products).
- Middlewares: Funções que processam requisições/respostas (ex: autenticação, logs).
- Velocidade: Leve e rápido para criar APIs ou servidores web.

Node.js Puro vs Express.js

Figura 1: Código exemplo em Node.js

```
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4    if (req.url === '/') {
5      res.end('Olá, mundo!');
6    } else {
7      res.end('Rota não encontrada!');
8    }
9  });
10
11 server.listen(3000);
```

Fonte: Elaboração própria.

Figura 2: Código exemplo em Express.js

```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Olá, mundo!');
6  });
7
8  app.listen(3000);
```

Fonte: Elaboração própria.

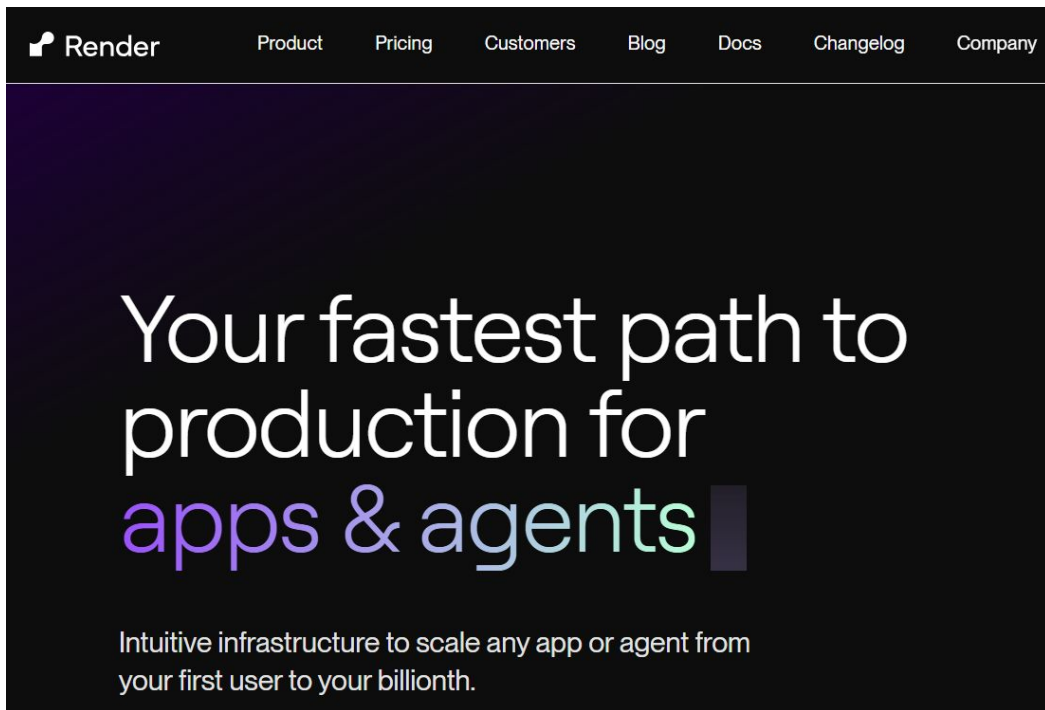
Quando usar Express.js?

- 1. Use Express.js para APIs REST eficientes, integração com bancos de dados e criação de backends escaláveis para aplicações web e mobile.
- 2. Ideal para servir páginas com templates e middleware que simplificam autenticação, logs e tratamento de erros.
- 3. Evite para aplicações em tempo real (prefira WebSockets) ou processamento pesado (use Workers), sendo melhor para prototipagem rápida.

Render para Simular Web Services

- Plataforma de hospedagem em nuvem moderna. Suporta Node.js, Python e outras linguagens. Integração fácil com repositórios Git.
- Oferece deploy contínuo automático. Planos gratuitos para projetos pequenos. Interface simples e intuitiva para iniciantes.
- Escalável para aplicações profissionais. Certificado SSL incluso gratuitamente. Ideal para APIs e microsserviços.

Render para Simular Web Services



Fonte: render.com

Benefícios do Render

- Deploy rápido em poucos cliques. Configuração simplificada sem terminal. Atualizações automáticas via GitHub.
- Ambiente de produção profissional. Escalabilidade automática sob demanda. Monitoramento de desempenho integrado.
- Suporte técnico eficiente. Infraestrutura confiável e segura. Perfeito para projetos acadêmicos.

Criando uma API REST com Express

- # Passo 1: Commit do Projeto no Github
 - Deixe disponível o projeto em um repositório do Github
- # Passo 2: Criando a conta no Render
 - Acesse dashboard.render.com
- # Passo 3: Crie novo "Web Service"
 - Clique no botão New e conecte o repositório no Github

Criando uma API REST com Express

- # Passo 4: Defina comando de start
 - Em Build Command: node / Em Start Command: node api.js
- # Passo 5: Deploy web service
 - Após realizar o deploy, utilize: seu-projeto.onrender.com

Atividade 01

1. Atualize o repositório da disciplina com todos os arquivos e alterações realizadas, garantindo que a versão disponibilizada esteja completa e corresponda à versão final da atividade.
2. Após atualizar o repositório, verifique se todos os arquivos foram enviados corretamente. Em seguida, copie o link do repositório atualizado e envie-o por meio do formulário abaixo, garantindo que o endereço informado esteja correto e acessível: <https://forms.gle/L7iL7jx1RzyiXBnk9>

Alterando o Projeto Express

Nos slides anteriores, fizemos uma recapitulação do que foi desenvolvido na aula passada. Agora, vamos evoluir o projeto: em vez de apenas exibir a data e hora de forma estática, construiremos uma aplicação com Express.js que permitirá armazenar, visualizar, editar e excluir notas, aplicando na prática o conceito de CRUD (Create, Read, Update, Delete).

Link: <https://notas-app-react.vercel.app/>

Alterando o Projeto Express

Para organizar a estrutura do projeto, utilizaremos um arquivo [JSON](#) como estratégia de armazenamento de dados, onde serão salvas todas as notas criadas pelo usuário. Além disso, teremos um arquivo [server.js](#), que será responsável por configurar e executar o servidor utilizando o framework Express.js. Esse servidor irá expor uma API RESTful com funcionalidades de CRUD (Create, Read, Update, Delete), permitindo criar novas notas, visualizar todas as existentes, atualizar o conteúdo de notas específicas e também removê-las.

Rotas CRUD da Aplicação

- **Listar Notas (GET /api/notes):** Retorna todas as notas armazenadas.
- **Criar Nota (POST /api/notes):** Adiciona uma nova nota. Exige os campos titulo e texto. Cada nota recebe um ID único (Date.now().toString()) e a data de criação.
- **Obter Nota por ID (GET /api/notes/:id):** Retorna uma nota específica pelo seu ID.
- **Atualizar Nota (PUT /api/notes/:id):** Atualiza uma nota existente pelo ID. Exige os campos titulo e texto.
- **Excluir Nota (DELETE /api/notes/:id):** Remove uma nota pelo ID. Retorna status 204 se a exclusão for bem-sucedida.

Criando uma API REST com Express - CRUD

- # Passo 1: Criando a pasta do projeto
 - Crie uma nova pasta chamada "projeto-notas" e abra no VS Code.
- # Passo 2: Instalando o [Express.js](#)
 - Execute no terminal: "npm install express"
- # Passo 3: Instalando o Body-Parser
 - Execute no terminal: "npm install body-parser" - biblioteca que permite ao servidor ler dados enviados no corpo das requisições HTTP, como JSON em métodos POST e PUT.
- # Passo 4: Criando o arquivo principal
 - Crie um arquivo chamado "server.js" e seguir os exemplos nos próximos slides.

Criando uma API REST com Express - CRUD

```
// Importa o Express, Body-Parser e FS
const express = require('express');
const bodyParser = require('body-parser');
const fs = require('fs');

const app = express();
const PORT = 3000;
const FILE = 'data.json';

// Permite receber JSON
app.use(bodyParser.json());

// Libera acesso externo (CORS)
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  next();
});
```


Criando uma API REST com Express - CRUD

```
// Função para ler arquivo
function readNotes() {
  try {
    const data = fs.readFileSync(FILE);
    return JSON.parse(data);
  } catch {
    return [];
  }
}

// Função para salvar arquivo
function saveNotes(notes) {
  fs.writeFileSync(FILE, JSON.stringify(notes, null, 2));
}

// GET - Listar notas
app.get('/api/notes', (req, res) => {
  const notes = readNotes();
  res.json(notes);
});
```

Criando uma API REST com Express - CRUD

```
// =====  
// POST - Criar nota  
// =====  
app.post('/api/notes', (req, res) => {  
  const notes = readNotes();  
  
  const novaNota = {  
    id: Date.now().toString(),  
    titulo: req.body.titulo,  
    texto: req.body.texto  
  };  
  
  notes.push(novaNota);  
  saveNotes(notes);  
  
  res.json(novaNota);  
});
```

Criando uma API REST com Express - CRUD

```
// =====  
// PUT - Editar nota  
// =====  
app.put('/api/notes/:id', (req, res) => {  
  const notes = readNotes();  
  
  const index = notes.findIndex(n => n.id === req.params.id);  
  
  if (index >= 0) {  
    notes[index].titulo = req.body.titulo;  
    notes[index].texto = req.body.texto;  
    saveNotes(notes);  
    res.json(notes[index]);  
  } else {  
    res.status(404).json({ erro: 'Nota não encontrada' });  
  }  
});
```

Criando uma API REST com Express - CRUD

```
// =====  
// DELETE - Excluir nota  
// =====  
app.delete('/api/notes/:id', (req, res) => {  
  const notes = readNotes();  
  
  const novasNotas = notes.filter(n => n.id !== req.params.id);  
  
  saveNotes(novasNotas);  
  
  res.json({ mensagem: 'Nota removida' });  
});  
// Inicia servidor  
app.listen(PORT, () => {  
  console.log('Servidor rodando em http://localhost:3000');  
});
```

Criando uma API REST com Express - CRUD

- # Passo 5: Criando o arquivo de dados
 - Crie um arquivo chamado "data.json" - Adicionar o exemplo abaixo

```
[ {  
  "id": "1",  
  "titulo": "Lembretes",  
  "texto": "Comprar leite e pão",  
  "criadoEm": "2026-04-28T10:00:00Z"  
},  
{  
  "id": "2",  
  "titulo": "Tarefas do trabalho",  
  "texto": "Enviar relatório até sexta-feira",  
  "criadoEm": "2026-04-28T10:00:00Z"  
}]
```

Criando uma API REST com Express - CRUD

- # Passo 6: Executando o servidor
 - Execute no terminal: "node server.js"
- # Passo 7: Criando o Front-end em React
 - Crie a interface React para listar, cadastrar, editar e excluir notas consumindo a API.
- # Passo 8: Publicando o Back-end no Render
 - Envie o projeto Express para o GitHub e faça o deploy no Render.
- # Passo 9: Conectando API no React
 - Copie a URL gerada no Render e substitua "http://localhost:3000/api/notes" no React pelo endpoint online.
- # Passo 10: Publicando o Front-end na Vercel
 - Envie o projeto React para o GitHub e faça o deploy na Vercel.

Atividade 02

1. Após desenvolver a aplicação utilizando Express.js, realize o deploy do back-end na plataforma Render, garantindo que a API esteja acessível online.
2. Em seguida, construa uma aplicação Front-End que consuma as rotas criadas, permitindo ao usuário realizar todas as operações CRUD (criar, visualizar, editar e excluir notas), interagindo diretamente com o arquivo JSON utilizado no back-end.

Atividade 02

3. Organize tudo em um documento contendo prints do código, da aplicação em funcionamento, o link do repositório no GitHub e o link do deploy realizado na Vercel. OBS: O Restante da atividade está contido nos próximos slides.

Referência:

- <https://github.com/devisontakatu/exemplo-api-backend>
- <https://github.com/devisontakatu/exemplo-api-frontend>

Questões para Refletir

- Quais são os riscos de segurança desse projeto e como poderiam ser eliminados ou mitigados?
- Usar um arquivo JSON (data.json) para armazenar notas é uma boa prática em produção? Quais são as vantagens e desvantagens dessa abordagem?
- Quais limitações um servidor baseado em arquivo JSON teria se o número de notas crescesse para 10.000 registros?
- O código atual está todo em server.js. Por que isso é problemático e como você melhoraria a organização da lógica?

DOCUMENTANDO UMA API

Documentando uma API

- Normalmente, as APIs são acompanhadas por uma documentação técnica que descreve em detalhes como utilizá-las.
- Essa documentação apresenta os endpoints disponíveis, os métodos HTTP suportados, os parâmetros exigidos, os formatos de resposta e exemplos de requisições.

Postman

- Postman é uma ferramenta colaborativa de desenvolvimento de APIs que simplifica o processo de criar, testar, documentar e monitorar requisições HTTP.
- Ele oferece uma interface intuitiva para enviar solicitações (GET, POST, PUT, DELETE, etc.), analisar respostas e automatizar testes, eliminando a necessidade de usar linhas de comando ou código complexo. Sua versatilidade o torna essencial para desenvolvedores, QA engineers e equipes de DevOps.

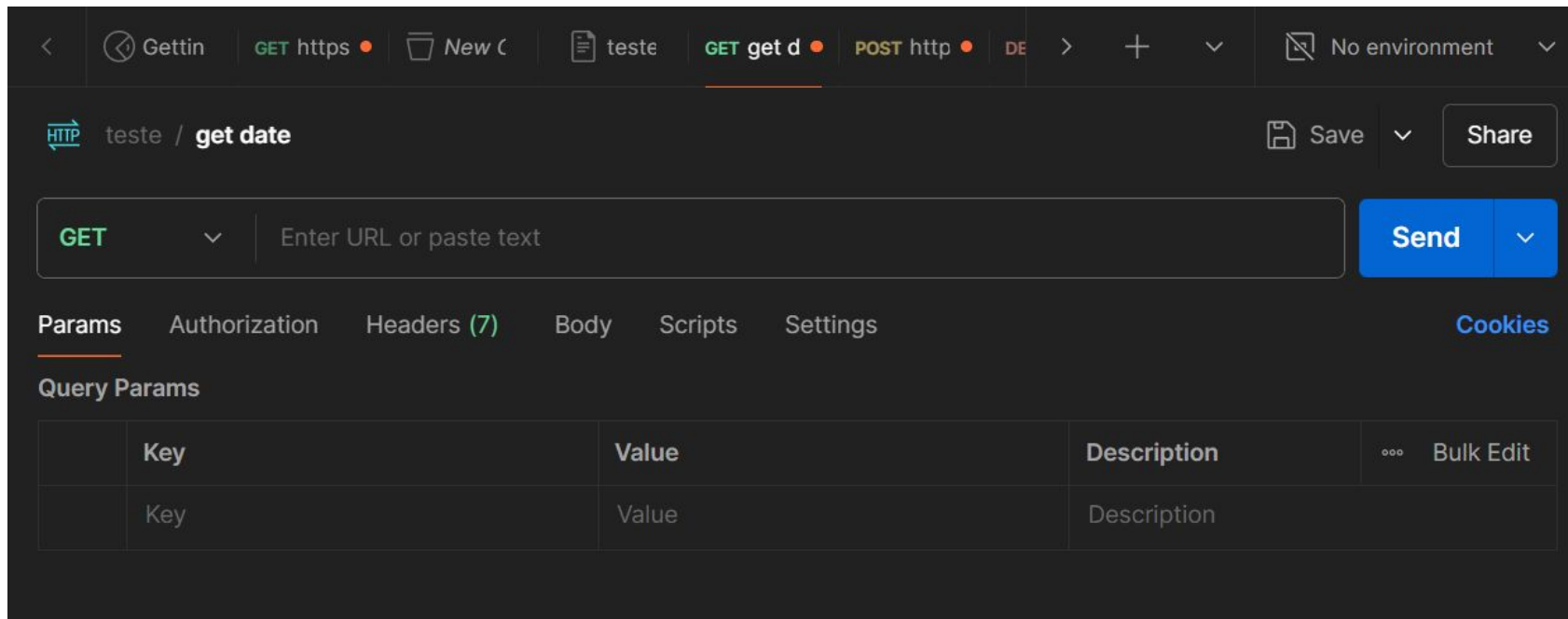
Postman

- Além de testes manuais, o Postman permite criar coleções de requisições, facilitando a organização e o compartilhamento de endpoints entre equipes.
- Com recursos como variáveis de ambiente, scripts pré-request e testes automatizados, ele ajuda a garantir a confiabilidade e a eficiência das APIs. Também é possível gerar documentação automática e integrá-lo a pipelines de CI/CD usando o Newman (CLI do Postman).

Postman

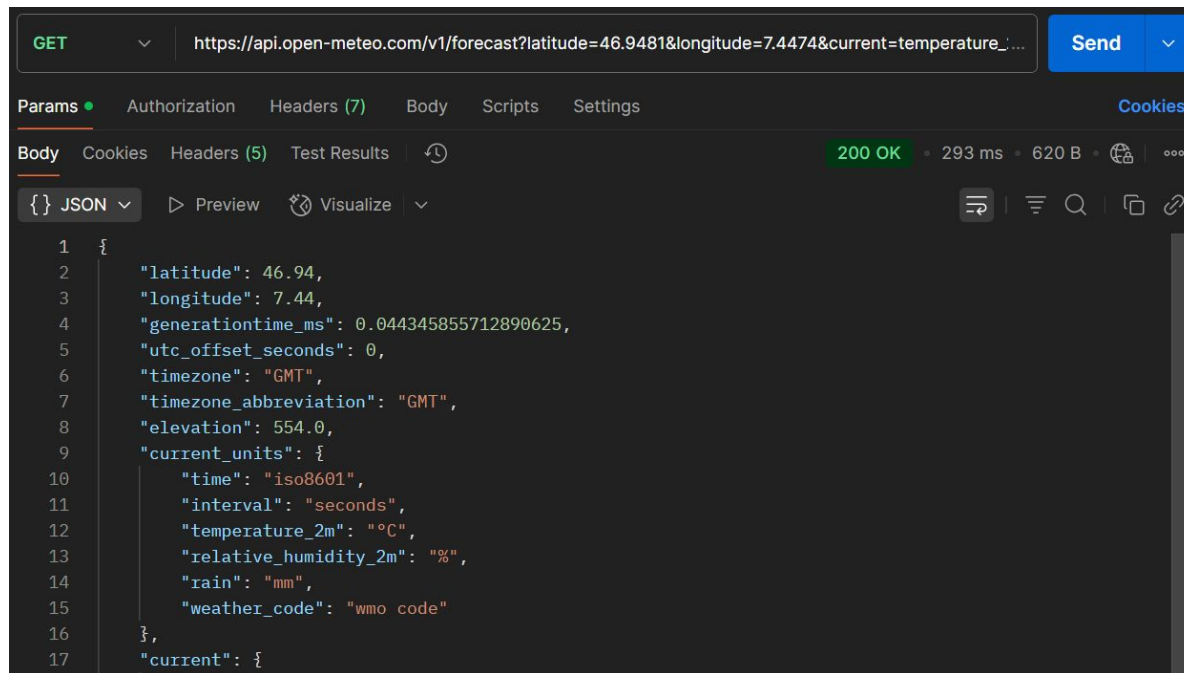
- O Postman vai além do básico, oferecendo recursos avançados como Mock Servers (para simular APIs antes da implementação), Monitoramento (para verificar a disponibilidade de endpoints) e suporte a GraphQL, WebSockets e autenticação OAuth 2.0.
- Sua versão gratuita já é poderosa, enquanto planos empresariais incluem colaboração em tempo real e gerenciamento de APIs em larga escala. Por sua facilidade e robustez, tornou-se um padrão no desenvolvimento moderno de software.

Postman



Fonte: <https://www.postman.com/>

Postman



Fonte: <https://www.postman.com/>

Atividade 02

1. Crie uma coleção no Postman documentando as quatro operações CRUD (Create, Read, Update, Delete) da API criada.
2. Configure as requisições HTTP e defina os parâmetros para utilizar cada endpoint com descrições claras sobre códigos de resposta.
3. Organize tudo em um documento contendo prints do código, da aplicação em funcionamento e o link da coleção do Postman. Envie o documento como entrega da atividade na plataforma CANVA.

Referências

1. SOUZA, Natan. ****Bootstrap 4: conheça a biblioteca front-end mais utilizada no mundo.**** São Paulo: Casa do Código, 2018. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
2. MACHADO, Kheronn Khennedy. ****Angular 11 e Firebase: construindo uma aplicação integrada com a plataforma do Google.**** São Paulo: Casa do Código, 2021. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 13 maio 2025.
3. EIS, Diego. ****Guia Front-end: o caminho das pedras para ser um dev front-end.**** São Paulo: Casa do Código, 2015. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
4. GONÇALVES, Edson. ****Desenvolvendo aplicações Web com JSP, Servlets, JavaServer Faces, Hibernate, EJB 3 Persistence e Ajax.**** Rio de Janeiro: Ciência Moderna, c2007.

Referências

5. HARTCOPP, Patrícia Ferreira. ****Métrica Web.**** São Paulo: Contentus, 2020. E-book (94 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/185191>. Acesso em: 30 abr. 2024.
6. NIEDERAUER, Juliano. ****Desenvolvendo Websites com PHP: aprenda a criar Websites dinâmicos e interativos com PHP e banco de dados.**** 3. ed. São Paulo: Novatec, 2017.
7. PREECE, J.; ROGERS, Y.; SHARP, H. ****Design de Interação: além da interação Homem-Computador.**** 3. ed. Porto Alegre: Bookman, 2013.
8. SOUSA, Roque Fernando Marcos. ****Canvas HTML 5: composição gráfica e interatividade na Web.**** Rio de Janeiro: Brasport, 2014. E-book (194 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/160686>. Acesso em: 22 jun. 2024.

Criando APIs para o Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br